

Session 4: Introduction to the tidyverse

Heiner Salomon

Bremen University

2026-05-11



Overview I

1 Introduction

2 Tidyverse

3 Tasks for the week

4 Any questions?



Introduction

Learning goals of today

- Short repetition from last week

Learning goals of today

- Short repetition from last week
- Discussing Reinhart-Rogoff

Learning goals of today

- Short repetition from last week
- Discussing Reinhart-Rogoff
- Introduction to tidyverse

Repetition

- Assignment: <-

Repetition

- Assignment: `<-`
- Commenting: `#`

Repetition

- Assignment: `<-`
- Commenting: `#`
- Sections: `# ---`

Repetition: Useful commands

- Importing data packages: haven (Stata, SPSS), readxl (Excel), readr (CSV)

Repetition: Useful commands

- Importing data packages: haven (Stata, SPSS), readxl (Excel), readr (CSV)
- Looking at your data: View()

Repetition: Useful commands

- Importing data packages: `haven` (Stata, SPSS), `readxl` (Excel), `readr` (CSV)
- Looking at your data: `View()`
- Summarizing data: `skimr::skim()`

Reading - Reinhart-Rogoff

- How did the mistake come about?

Reading - Reinhart-Rogoff

- How did the mistake come about?
- What are potential ways to prevent you doing similar mistakes?

Tidyverse

What will we learn today

- What is the tidyverse?

What will we learn today

- What is the tidyverse?
- “Piping”



What will we learn today

- What is the tidyverse?
- “Piping”
- `mutate()`

What will we learn today

- What is the tidyverse?
- “Piping”
- `mutate()`
- `filter()`

What will we learn today

- What is the tidyverse?
- “Piping”
- `mutate()`
- `filter()`
- `select()`

What is the tidyverse?

- tidyverse is many packages useful for typical data analysis

What is the tidyverse?

- tidyverse is many packages useful for typical data analysis
- Built on base R, tries to be more legible

What is the tidyverse?

- tidyverse is many packages useful for typical data analysis
- Built on base R, tries to be more legible
- Important packages (for me) are: ggplot2, dplyr, tidyr, stringr...

What is the tidyverse?

- tidyverse is many packages useful for typical data analysis
- Built on base R, tries to be more legible
- Important packages (for me) are: ggplot2, dplyr, tidyr, stringr...
- For more information: <https://tidyverse.org/>

Before we continue - tidylog()

- tidyverse commands very “quiet”

Before we continue - tidylog()

- tidyverse commands very “quiet”
- too easy to miss important changes to data

Before we continue - tidylog()

- tidyverse commands very “quiet”
- too easy to miss important changes to data
- tidylog() changes that and is extremely useful

Before we continue - tidylog()

- tidyverse commands very “quiet”
- too easy to miss important changes to data
- tidylog() changes that and is extremely useful
- Add tidylog at end of beginning of each script packages loaded in library()

“Piping” - the most important concept of the tidyverse

- One of the most important concepts

“Piping” - the most important concept of the tidyverse

- One of the most important concepts
- A pipe is an operator: `|>` (or sometimes `%>%`)

“Piping” - the most important concept of the tidyverse

- One of the most important concepts
- A pipe is an operator: `|>` (or sometimes `%>%`)
- Follows the logic of steps (first step `|>` second step `|>` third step)

“Piping” - the most important concept of the tidyverse

- One of the most important concepts
- A pipe is an operator: `|>` (or sometimes `%>%`)
- Follows the logic of steps (first step `|>` second step `|>` third step)
- usually starts with data set followed by function

Functions - basics

- Short detour on functions

```
# Understand standard arguments for head()
?head()

# Identical operations
head(iris, 10) # Order determines arguments
head(n = 10, x = iris) # arguments specified
```

rsität
en

Functions - basics

- Short detour on functions
- Functions in R comprised of different “arguments” separated by ,

```
# Understand standard arguments for head()
?head()

# Identical operations
head(iris, 10) # Order determines arguments
head(n = 10, x = iris) # arguments specified
```

rsität
en

Functions - basics

- Short detour on functions
- Functions in R comprised of different “arguments” separated by ,
- Order of arguments matters (or have to specify arguments)

```
# Understand standard arguments for head()  
?head()
```

```
# Identical operations  
head(iris, 10) # Order determines arguments  
head(n = 10, x = iris) # arguments specified
```

rsität
en

“Piping” - relation to functions

- In Base R, nesting of functions visually very confusing

“Piping” - relation to functions

- In Base R, nesting of functions visually very confusing
- Piping replaces first argument in next function with last step in piping process

“Piping” - relation to functions

- In Base R, nesting of functions visually very confusing
- Piping replaces first argument in next function with last step in piping process
- As result, “piped” code easier to understand and fix

“Piping” - a visual explanation

- Piping is adding each former step as first argument in next function

```
# "Piped" code (as in the tidyverse)
iris_data <- iris |>
  filter(Species = "virginica") |>
  mutate(Petal.Length = Petal.Length + 1)

# Spelled out in brackets (as in Base R)
iris_data <- mutate( filter(iris, Species = "virginica"),
                     Petal.Length = Petal.Length + 1)
```

Universität
Bremen

“Piping” - an example

```
# Step 1: Load a data set
iris_data <- iris
# Step 2: Filter the data
iris_data <- filter(iris_data, Species = "virginica")
# Step 3: Add one to each Petal.Length
iris_data <- mutate(iris_data, Petal.Length =
                    Petal.Length + 1)

# All in one go
iris_data <- iris |>
  filter(Species = "virginica") |>
  mutate(Petal.Length = Petal.Length + 1)
```

rsität
en

mutate() - Variable manipulation

- Often you will need to create new variables or change existing ones

mutate() - Variable manipulation

- Often you will need to create new variables or change existing ones
- Create new variable: `mutate(data, new_variable = old_variable)`

mutate() - Variable manipulation

- Often you will need to create new variables or change existing ones
- Create new variable: `mutate(data, new_variable = old_variable)`
- `old_variable` and `new_variable` can be the same

mutate() - Variable manipulation

- Often you will need to create new variables or change existing ones
- Create new variable: `mutate(data, new_variable = old_variable)`
- `old_variable` and `new_variable` can be the same
- At the right between “`old_variable`” and the “`)`”, you can add anything to manipulate the relationship



mutate() - Variable manipulation (code)

```
# Check the summary of Petal.Length before
iris_data |>
  select(Petal.Length) |>
  summary()

# Add 1 to each value of Petal.Length
iri_data <- iris |>
  mutate(Petal.Length = Petal.Length + 1)

# Check the summary of Petal.Length after
iris_data |>
  select(Petal.Length) |>
  summary()
```

rsität
en

Test: mutate() - Variable manipulation

- load your data sets

Test: mutate() - Variable manipulation

- load your data sets
- use mutate() to create a new variable

mutate() - Conditional manipulation

- Often you only need conditional manipulation

mutate() - Conditional manipulation

- Often you only need conditional manipulation
- Can be achieved combining mutate() with case_when()

mutate() - Conditional manipulation

- Often you only need conditional manipulation
- Can be achieved combining mutate() with case_when()
- `case_when(condition1 ~ new_value1, condition2 ~ new_value2,...)`

mutate() - Conditional manipulation (code)

```
# Check the distribution of Petal.Length before
iris |>
  select(Petal.Length) |>
  table()
# Using a simple binary case_when()
iris_data <- iris |>
  mutate(Petal.Length.binary =
    case_when(
      Petal.Length < 2 ~ "Small",
      Petal.Length >= 2 ~ "Large"
    ))
# Check the distribution of Petal.Length after
iris |>
  select(Petal.Length) |>
```

rsität
en

Test: mutate() - Conditional manipulation

- use mutate() with case_when() to create a new variable based on conditions found in another variable

filter() - Filtering “rows”

- Often data sets contain a lot of data

filter() - Filtering “rows”

- Often data sets contain a lot of data
- too much data might slow down operations and be confusing

filter() - Filtering “rows”

- Often data sets contain a lot of data
- too much data might slow down operations and be confusing
- filter() filters out unnecessary rows

filter() - Filtering “rows”

- Often data sets contain a lot of data
- too much data might slow down operations and be confusing
- filter() filters out unnecessary rows
- requires the ==, not only one =

filter() - Filtering “rows”

- Often data sets contain a lot of data
- too much data might slow down operations and be confusing
- filter() filters out unnecessary rows
- requires the ==, not only one =
- Usually, the ! indicates a reversal

filter() - Filtering “rows” (code)

```
# Check the number of rows before
nrow(iris_data)

# Filter out all species not "Virginica"
iris_data <- iris |>
  filter(Species == "virginica")

# Check the number of rows after
nrow(iris_data)
```



select() - Filtering “columns”

- select() is the same as filter(), but for columns (variables)

select() - Filtering “columns”

- select() is the same as filter(), but for columns (variables)
- uses “tidy selection”, such as :, c() etc.

select() - Filtering “columns”

- select() is the same as filter(), but for columns (variables)
- uses “tidy selection”, such as :, c() etc.
- for more information, check:
<https://dplyr.tidyverse.org/reference/select.html>

select() - Filtering “columns”

- select() is the same as filter(), but for columns (variables)
- uses “tidy selection”, such as :, c() etc.
- for more information, check:
<https://dplyr.tidyverse.org/reference/select.html>
- Usually, the ! indicates a reversal

select() - Filtering “columns” (code)

```
# Check the variables before
names(iris_data)

# Select all variables in between Petal.Length and Species
iris_data <- iris |>
  select(Petal.Length:Species)

# Check the variables after
names(iris_data)
```

Test: select() - Filtering “columns”

- use select() to reduce the number of columns in your data set

Test: select() - Filtering “columns”

- use select() to reduce the number of columns in your data set
- please make sure to not assign the old data set name to not “lose” the data

Code formatting

- Code formatting seems tedious

Code formatting

- Code formatting seems tedious
- But formatting is important for clarity

Code formatting

- Code formatting seems tedious
- But formatting is important for clarity
- Formatting speaks to line breaks, indentation and spacing

Code formatting

- Code formatting seems tedious
- But formatting is important for clarity
- Formatting speaks to line breaks, indentation and spacing
- For tidyverse style guide, see <https://style.tidyverse.org/>

Code formatting - using `styler()`

- You can also use a package for formatting, `styler()`

Code formatting - using styler()

- You can also use a package for formatting, `styler()`
- It also includes a RStudio plugin

Code formatting - using styler()

- You can also use a package for formatting, `styler()`
- It also includes a RStudio plugin
- You can find it here: <https://styler.r-lib.org/>

Some useful hotkey tips

- RStudio has several very useful hotkeys, for instance:

Some useful hotkey tips

- RStudio has several very useful hotkeys, for instance:
- Bulk - commenting: “Comment out” larger chunks of R-code
-> Ctrl + Shift + C

Some useful hotkey tips

- RStudio has several very useful hotkeys, for instance:
- Bulk - commenting: “Comment out” larger chunks of R-code
-> Ctrl + Shift + C
- Multi-line cursor: Change code in multiple lines
simoultaneously -> Click beginning line + Hold Alt + Shift
and click end line

Tasks for the week

Tasks for the week

- Please don't forget to send me progress updates by Friday
15:00

Tasks for the week

- Please don't forget to send me progress updates by Friday 15:00
- This time we will use email

Reading for next week

Munafò, Marcus R., Brian A. Nosek, Dorothy V. M. Bishop, et al. 'A Manifesto for Reproducible Science'. Nature Human Behaviour 1, no. 1 (2017): 0021.

<https://doi.org/10.1038/s41562-016-0021> - We will discuss those ideas at the beginning of next week's class

Any questions?